



南京邮电大学
Nanjing University of Posts and Telecommunications

文献阅读汇报

A generic communication scheduler for distributed DNN training acceleration

面向分布式DNN训练加速的通用通信调度器

汇报人：刘品月

2026年3月21日

论文背景与动机

论文背景

深度神经网络（DNN）使用广泛，但由于数据量变大和DNN模型规模增长，训练DNN模型耗时严重。最常见扩展和加速DNN训练的方式是数据并行，但由于**通信开销**，它的表现往往远非线性速度提升。

现有方案A：加速“单条”消息

代表技术： 用RDMA替换TCP，或者用NCCL库进行高效的All-reduce。

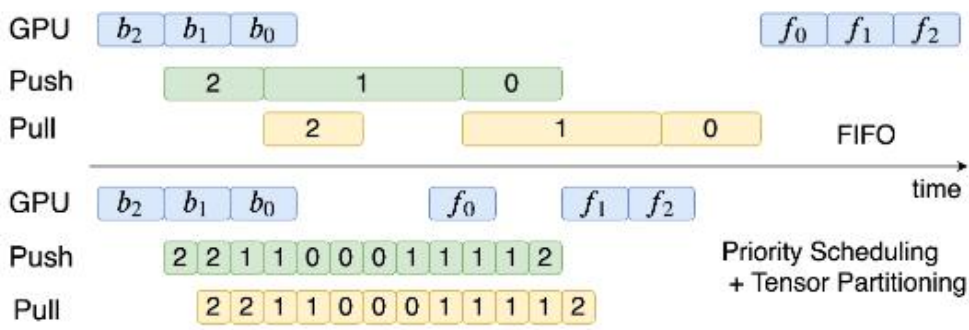
局限性： 这些是微观层面的优化，它们只是让每一条消息跑得更快，但没有改变消息之间的发送顺序。

现有方案B：通信调度

代表设计： P3和 TicTac

核心理念： 在现有深度学习框架中，通信操作默认按照FIFO的顺序执行，而通信调度通过改变不同层的通信顺序，来隐藏通信延迟。

局限性： 他们通常只针对特定深度学习框架（如MXNet或TensorFlow）和特定通信架构（如参数服务器），**难以迁移**到其他框架或通信方式（如all-reduce、RDMA）。



ByteScheduler 整体架构

ByteScheduler 的设计核心是实现与框架无关、与通信方式无关的通用调度层。其整体架构分为两部分：

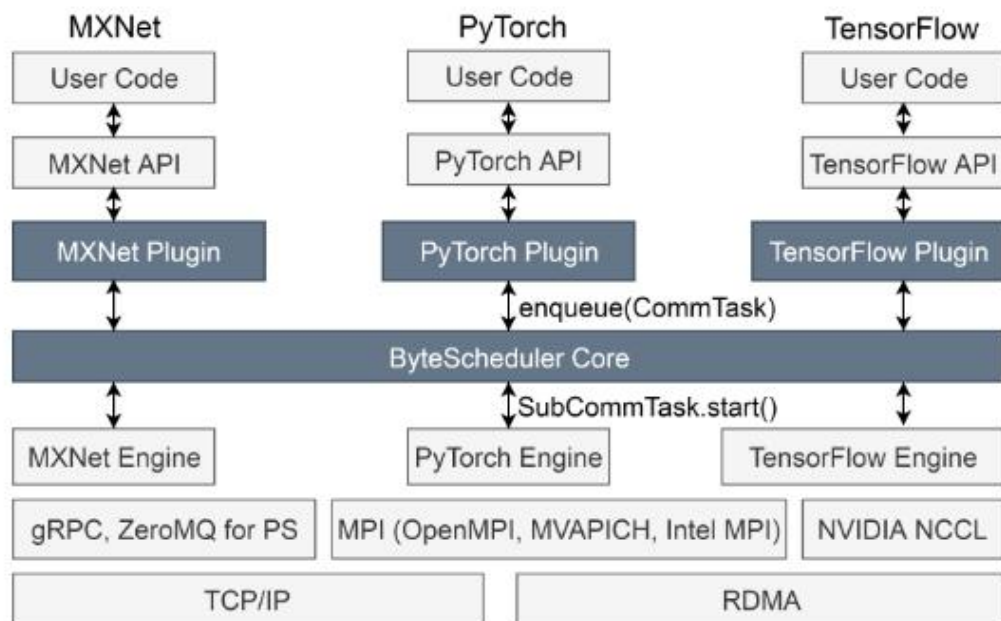
插件层：

- 针对每种机器学习框架（TensorFlow/PyTorch/MXNet）写一个**轻量插件**
- 插件的作用：把框架原生的通信操作**封装成统一的CommTask抽象**，并注入**依赖代理**，以便在不修改框架引擎的前提下与计算图交互

核心调度器：

一个独立于框架的模块，负责接收所有 CommTask，执行：

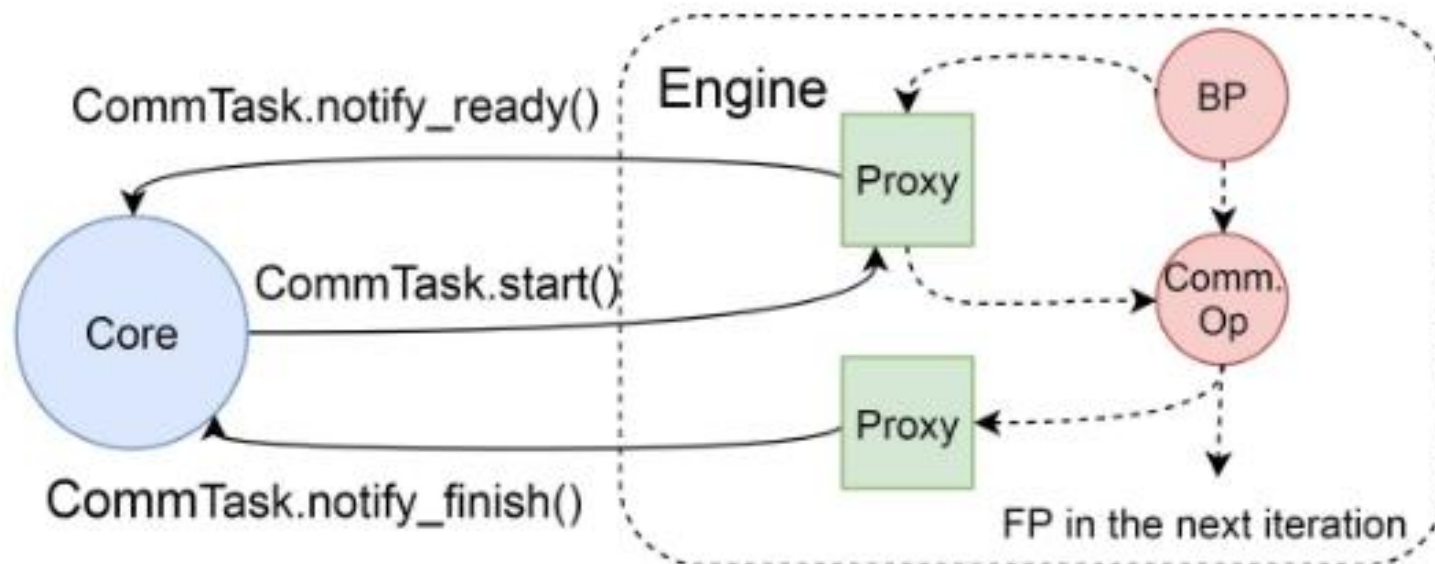
- **Tensor切分**：把大tensor切成小块
- **优先级排序**：按调度算法决定谁先发
- **发送控制**：通过credit-based滑动窗口控制发送时机



Dependency Proxy机制

Proxy

Proxy是ByteScheduler创建的一个操作。它可以发布到框架引擎中，并声称从/到其他操作的依赖关系。声明式引擎都提供了直接的API来定义这种依赖关系，因为这是此类引擎的基本特征。

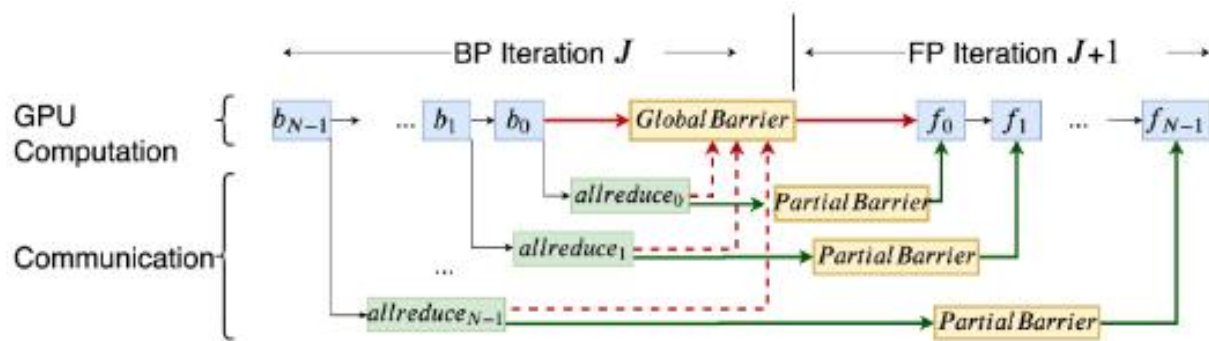
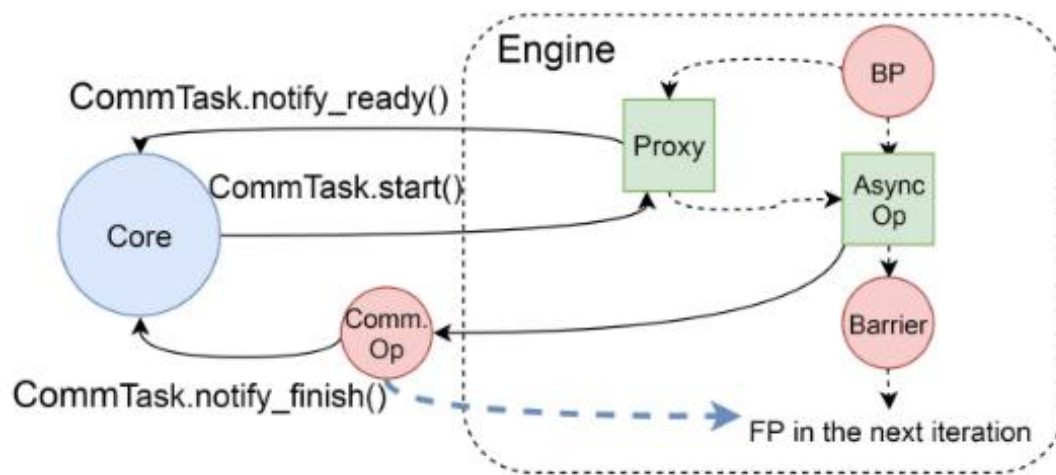


跨越global barrier

一些框架在连续两次迭代之间引入了一个global barrier。这使得调度在很大程度上是无效的——global barrier等待所有通信操作完成后再进行下一次迭代，因此改变通信顺序无关紧要。

解决方案

1. 将实际的通信操作替换为**异步操作**：异步操作将在后台启动实际通信，并立即返回，让全局屏障通过。换句话说，实际的通信操作是在引擎和计算图之外，通过Core，甚至在栅栏之后运行的。在每次实际通信操作结束后，我们添加一个回调来通知Core完成相关操作。
2. **重建依赖**：在下一轮迭代的每一层前向计算之前，插入一个Proxy，这个Proxy会阻塞，直到Core通知它“上一轮对应层的通信已完成”，只有收到通知，前向计算才能开始。



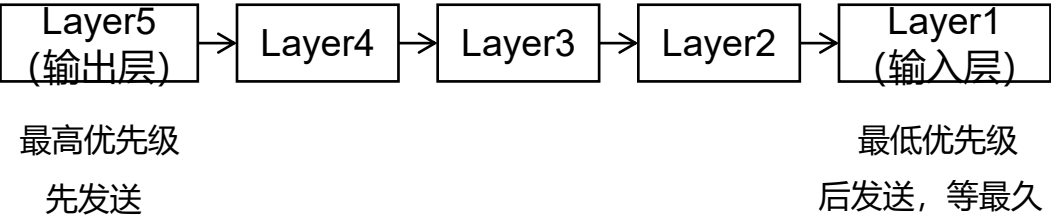
对比

场景	无barrier框架（MXNet）	有barrier框架（Tensorflow/PyTorch）
机制	Dependency Proxy直接控制通信	异步操作+层间Proxy重建依赖
通信位置	在计算图内	移除计算图，后台执行
下一轮开始时机	通信可与计算重叠	通信与计算完全重叠
正确性保障	框架原有依赖	ByteScheduler新增依赖

调度策略：优先级队列

通信顺序为什么重要？

在默认FIFO顺序下，后向传播产生的梯度是从最后一层到第一层依次发送的（因为BP是从输出层向输入层计算）。
但问题是：下一轮迭代的前向传播是从第一层开始的。layer1必须等它的参数pull完成才能开始计算。如果layer1的通信被排在最后，下一轮的前向计算就会被延迟。
结论：要想让计算和通信更好地重叠，应该让靠近输入层的通信优先执行。

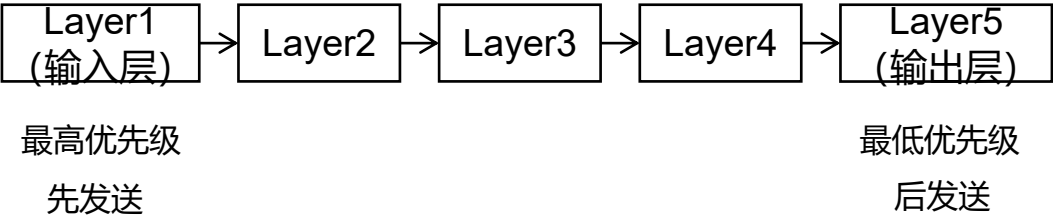


FIFO 顺序

ByteScheduler的调度策略：优先级队列

通信架构	优先级规则
参数服务器（PS）	push和pull都按层号排序：层号越小，优先级越高
All-reduce	同理：层号越小，all-reduce优先级越高

论文定理1证明，在理想条件下，这种优先级队列策略是最优的，能最大化计算和通信的重叠。



ByteScheduler 顺序

调度策略：Tensor切分

现实中问题：tensor大小差异巨大

以VGG16为例：

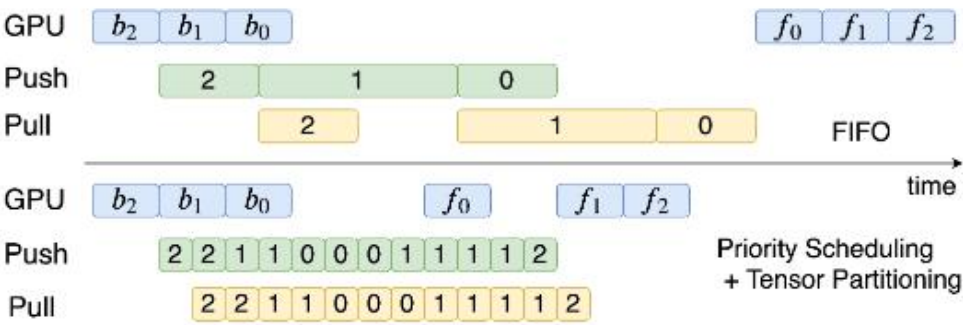
最小的tensor只有 256B

最大的tensor超过 400MB

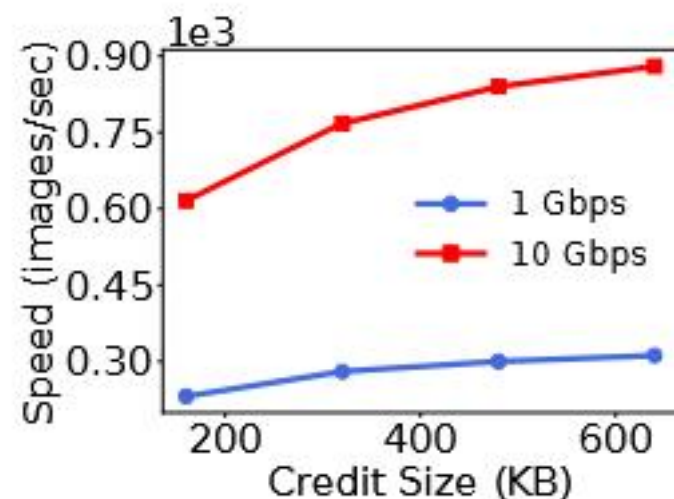
如果一个大tensor（比如来自某层的巨型梯度）已经开始发送，后面来的高优先级小tensor（来自更浅层的）就只能干等着，直到这个大tensor发完——即使它的优先级更高。

解决方案：Tensor切分

优点	说明
实现抢占	小块发完就可以让给高优先级任务，不必等整个tensor传完
提升双向带宽利用率（PS架构）	切分后，push和pull可以流水线式并行，而不是串行
缓解PS负载不均	大tensor被切分后，可以更均匀地分配到多个PS上



调度策略：基于信用的滑动窗口机制



(b) Different credit sizes

核心思想：

- 维护一个信用额度（credit），表示当前可以发送多少数据
- 只有信用足够时，才能发送新的tensor小块
- 发送后信用减少，小块传输完成（收到确认）后信用恢复

credit大小	带宽利用率	抢占及时性
太小	低，网络经常空等	高，随时可插队
太大	高，网络始终忙碌	低，高优先级块要等窗口里的低优先级块发完

调度算法

Algorithm 1 The scheduling algorithm in Core

Input:

```
commtasks ← a set of unready SubCommTasks
queue ← priority queue of SubCommTasks
credit ← credit size for credit-based preemption
unit ← partition size that is used to partition a CommTask
1: /* Partition a CommTask in Core */
2: procedure PARTITION(task)
3:   subtasks = task.partition(unit)
4:   commtasks.add(subtasks)
5: /* Enqueue a ready SubCommTask based on priority */
6: procedure READY(priority, subtask)
7:   queue.put(priority, subtask)
8:   commtasks.remove(subtask)
9: /* Return credits when a SubCommTask is finished */
10: procedure FINISH(subtask)
11:   credit = credit + subtask.tensor.size
12: /* Schedule SubCommTasks in the priority queue */
13: procedure SCHEDULE
14:   while True do
15:     priority, subtask = queue.get()
16:     if credit > subtask.tensor.size then
17:       credit = credit - subtask.tensor.size
18:       subtask.start()
19:     else
20:       wait until a subtask is finished and returns credits
```

算法流程分四步：

第一步：当一个通信任务到达时，Core会调用partition()把它切成多个SubCommTask，然后按优先级入队。

第二步：有一个调度线程不停地从队列头部取最高优先级的任务。拿到任务后，它会检查当前剩余信用是否足够发送这个tensor小块。

第三步：如果信用足够，就调用start()发送，同时信用减少；如果信用不够，线程就等着，直到有任务完成返还信用。

第四步：当一个小块传输完成，会调用finish()回调，信用增加，同时唤醒等待的调度线程，继续处理下一个任务。

优化部分：贝叶斯优化方法自动寻找最优参数

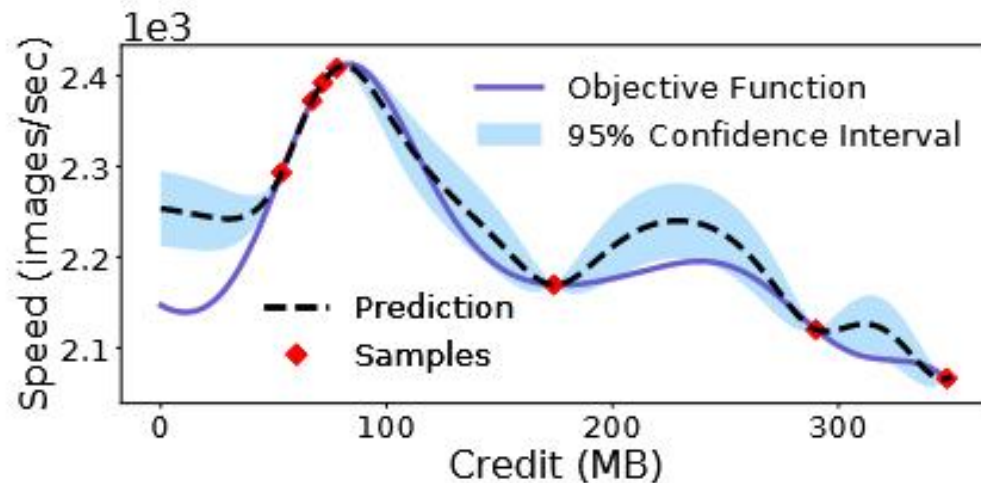
切分大小、信用大小这两个参数的最优值是不固定的。

常规调参方法不适用：

网格搜索，即枚举所有可能的信用等级和分区大小的组合，代价太高。

带动量的SGD，由于它是非参数的，无法得到曲线的精确梯度。

通过实验，最终确定使用**贝叶斯优化(BO)**来共同调整信用和切分大小。



1. 先试几个点：随机选几组，记录训练速度
2. 拟合一个“猜测的曲线”：用高斯过程回归，拟合出性能函数，并给出置信区间
3. 决定下一个试哪里：用“Expected Improvement”规则，选择最有可能比当前最好结果更优的点
4. 重复：试新点，更新模型，再选下一个点

实验结果

实验设置:

16台物理器, 每个物理机有64个CPU核, 256GB内存, 8个Tesla V100 GPU, 带宽100Gbps

基准:

2个CNN模型VGG16和ResNet50网络, 以及1个RNN模型Transformer作为基准模型。

结果:

所有实验结果都证明: ByteScheduler 能在所有主流框架、所有通信架构、所有传输协议下, 实现显著的训练加速。

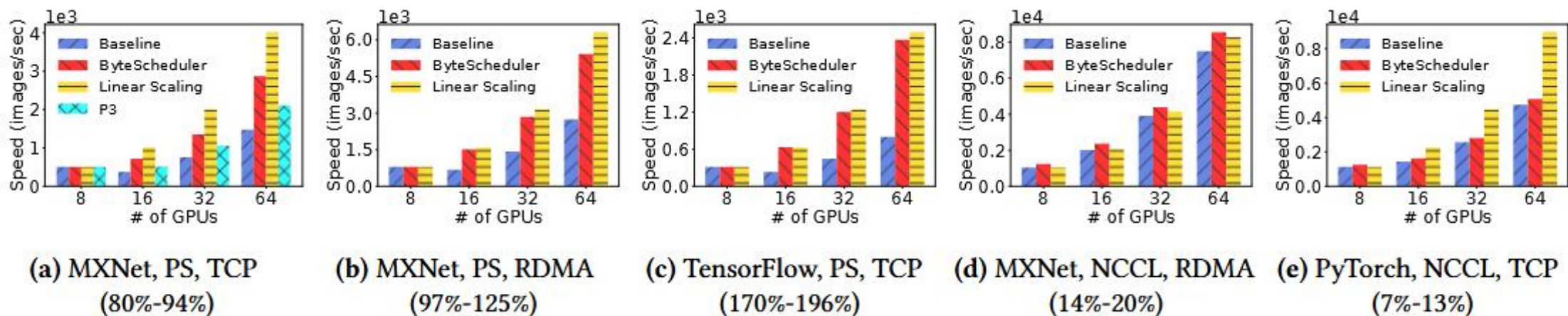


Figure 10. Training VGG16 model. The numbers in parentheses are ByteScheduler speedup percentages as compared with the baseline.

该论文与我课题的关系

2、应用感知的集合通信传输机制

针对 AI 大模型训练中计算通信过程复杂、通信模式多样、服务级别协议差异化的问题，研究应用感知的集合通信传输机制。首先，对 AI 大模型训练的参数同步过程中的稀疏梯度、高精度优化器状态等进行稀疏编码和压缩；其次，对数据并行、流水线并行、张量并行等不同阶段和模式中的传输需求进行分类和抽象，联合设计计算和通信的整体过程，实现计算和通信的相互掩盖；最后，感知 AI 大模型训练过程中的通信数据使用先后顺序，识别业务流量优先级，并通过差异化的流量调度，实现对高优先级流的优先传输和带宽保证。

Dependency Proxy：如何在不侵入框架的情况下插入控制逻辑，ByteScheduler 通过轻量级插件 + Proxy 的方式，在不修改框架引擎的前提下，实现了对通信操作的精准控制，这种设计思路，为我课题提供了重要参考，让我能在保证框架兼容性的同时，实现计算和通信的协同调度。

基于信用的滑动窗口：ByteScheduler 通过信用额度控制网络带宽的分配，在保证优先级的同时充分利用带宽，这种带宽管理的思路，可以被借鉴到课题的差异化调度中。



南京邮电大学
Nanjing University of Posts and Telecommunications

文献阅读汇报

请老师批评指正！

汇报人：刘品月

汇报时间：2026年3月21日